

AUDITING STRUCTURED POLICY LEARNING FOR STREAMING TRANSACTION COLLATERAL CONTROL: FACTORIZATION, CONDITIONING, AND CROSS-SCALE TRANSFER

Yingda Yu¹, Sijia Zhou¹, Jiaqi Xuan¹, Zhentong Ye², Guanchao Tong^{1,*}

¹Wenzhou-Kean University

²Northwestern University

*Corresponding author

ABSTRACT

Off-chain payment nodes route a stream of transactions into k collateralized wallets and periodically pay to flush a wallet. We treat the resulting collateral-control task as an online sequential decision problem and audit, in a controlled multi-seed reimplementation, which structural inductive biases help a learned PPO controller. Three questions organize the study: whether factoring the joint (settle, flush) action into two heads improves on a quadratic joint head; whether conditioning the flush head on the planned settle action adds value; and whether a permutation-equivariant, k -independent encoder enables deployment across wallet counts. Across five seeds and four collateral budgets, factorization is the reliable effect: the settle-conditioned factored policy beats the joint head at every budget and the independent variant at most budgets, whereas zero- and shuffled-conditioning ablations detect no benefit from the settle-to-flush path. The equivariant encoder loads and runs zero-shot at unseen wallet counts, with asymmetric transfer and one training failure at the largest k . A single validation-selected balance-feedback rule remains the strongest tested controller, both on stationary streams and across abrupt regime switches.

Index Terms— reinforcement learning, structured policies, resource allocation, permutation equivariance, online decision making.

1. INTRODUCTION

Off-chain payment networks such as the Lightning Network [1, 2, 3] process streams of transactions over wallets backed by finite collateral, and routing under scarce channel funds has been studied for payment channel networks [4]. Transactions arrive sequentially, each with a requested value, and at every arrival the node must either settle it into one of k wallets that currently holds enough free balance or drop it; independently, it may *flush* one wallet, paying a fixed fee to empty and recharge it while the wallet briefly cannot accept funds. The problem is inherently sequential: a flush frees capacity for future arrivals but costs money and removes a wallet from service, while greedily accepting now can starve a later, larger transaction. Collateral is partitioned per wallet rather than pooled, and future arrivals are unknown. Such sequential decisions are the domain of online competitive analysis [5] and its learning-augmented variants [6]; the closest formal model, online collateral maintenance with k wallets of capacity C/k , admits worst-case competitive guarantees [7], alongside related online-inventory results [8].

This structure makes a naive policy parameterization costly. A joint categorical controller emits one logit per (settle, flush) pair,

$(k+1)^2$ outputs at $k=24$, allocating capacity to pairs that are conflicting or dominated even though the two decisions are only weakly coupled. Branched and factorized action heads address precisely this issue [9]. Factorization is therefore the natural structural move: two smaller $k+1$ -way heads reduce the output count to $2(k+1)$. A residual question is whether the coupling between the two choices matters—specifically, whether conditioning the flush decision on the *planned* settle wallet (a directed settle→flush information path) adds value. We treat this mechanism as a hypothesis to test rather than a feature to assume.

A second structural question arises at deployment. The wallet count k is an operating condition that can change between deployments, yet a flat multilayer perceptron bakes k into its weight shapes: a model trained at one k cannot be loaded at another. Encoding the wallets as an unordered set with a shared, permutation-equivariant function removes all k -dependent parameters and allows one trained policy to be deployed zero-shot at a new wallet count. Whether such portability carries a matched-scale accuracy cost is, again, an empirical question.

Finally, learned controllers should be assessed against a baseline that uses the most informative available signal, namely current wallet fill. We therefore include a validation-selected balance-feedback rule on equal footing with the learned policies, and structure the paper as an audit of which structural inductive biases help on this task: does factorization help, does settle-conditioning add value, does an equivariant encoder provide cross- k deployability, and how does learning compare with a simple feedback rule? Questions are answered on a faithful port of the prior environment and its twelve-regime traffic generator with matched seeds and paired confidence intervals; null results are reported together with positive ones.

Our contributions are:

- A controlled five-seed audit of joint versus factorized PPO action parameterizations (joint head, independent heads, settle-conditioned heads), including matched zero/shuffle mechanism ablations, on a faithful, tested port of the environment and the twelve-regime generator.
- A permutation-equivariant set encoder with no k -dependent parameter (in the spirit of Deep Sets [10]), which loads and runs zero-shot at unseen wallet counts where a fixed-shape flat MLP cannot reuse its weights, together with an explicit account of its transfer asymmetry.
- A comparison against a validation-selected balance-feedback baseline on both stationary streams and abrupt regime switches, including per-seed paired tests and Holm-corrected significance statements.

2. STREAMING COLLATERAL CONTROL

Collateral $C \in \{800, 900, 1000, 1200\}$ is split into $k=24$ homogeneous wallets of capacity C/k ; transfer experiments additionally use $k \in \{6, 12\}$. At each of $T=1000$ decisions a transaction of size $x_t \in [0, 1000]$ arrives. Wallet i is described by $z_{i,t} = (\text{balance}_i, \text{availability}_i, \text{cooldown}_i)$, and two global scalars $g_t = (\bar{x}_t, t/T)$ give the normalized transaction size and time progress; the observation is the concatenation, of dimension $3k+2$. The controller chooses $a_s, a_f \in \{1, \dots, k\} \cup \{\perp\}$, a settle wallet and a flush wallet (null = no-op). A flush executes *first*: the selected wallet is charged, made unavailable for $F=3$ decisions, and then refilled to full; the transaction is settled afterwards, accepted if a_s is usable and holds free balance at least x_t , and dropped otherwise. Flushing and settling the same wallet is not forbidden, but the flushed wallet is in cooldown when settlement is attempted, so such conflicting pairs typically drop the transaction. Drops are attributed as *oversize* ($x_t > C/k$, infeasible for any policy), *frozen* (settle wallet in cooldown), or *avoidable* (insufficient balance or a flush/settle conflict that proactive flushing could have prevented). Per-episode Money is

$$M = p \sum_t x_t \mathbb{I}[\text{accepted at } t] - \tau \sum_t \mathbb{I}[a_{f,t} \neq \perp], \quad (1)$$

with acceptance price $p=1$ and flush fee $\tau=10$; a drop carries no additional penalty. A stochastic policy is optimized with episodic PPO [11] and generalized advantage estimation [12], and evaluated deterministically (argmax) on fixed episode pools. Episodes are generated with fixed seeds from twelve regimes spanning steady, large-transaction, and correlated-burst traffic: 5000 training, 300 validation, and 200 test episodes per regime. Every method and seed uses the same deterministic test pools; the statistical unit is the matched training seed ($n=5$ main, $n=3$ ablations/switching), not the individual test episodes. All design choices use validation only, and failed seeds are retained (Fig. 1).

3. POLICIES

JA-PPO emits one categorical over all settle/flush pairs, $(k+1)^2$ logits (625 at $k=24$). It treats the step as one large joint decision, which inflates the output layer and assigns probability mass to conflicting or dominated pairs. Structured large-action spaces have been addressed with action embeddings [13], parameterized-action hierarchies [14], action branching [9], and neighborhood construction [15].

IFAC factors the action into two independent $k+1$ categoricals, $\pi(a_s, a_f | s) = \pi_s(a_s | s) \pi_f(a_f | s)$ (50 logits), sharing a trunk. This removes the quadratic head, encodes the weak-coupling prior between the sub-decisions, and lets each head learn from its own signal at every step instead of from rarely observed joint pairs.

SC-FAC keeps the factorization and adds a directed settle→flush path. At the *policy level* a settle action a_s is first sampled (or selected, under deterministic evaluation); an embedding $e(a_s)$ of that planned action then conditions the flush distribution, $\pi_f(a_f | s, a_s)$. The intended mechanism is coordination—in particular, avoiding a flush of the wallet selected for the imminent settlement and choosing complementary flush/settle pairs. This conditioning is a dependency within the policy, not an execution order: the environment still executes a_f before a_s , and no value has been moved when a_f is chosen. Two matched ablations share the architecture exactly: **no-cond.** zeroes $e(a_s)$ and **shuffled** supplies a deranged (wrong-wallet) embedding.

Set encoder. The wallets are intrinsically an unordered set. A shared function first encodes each wallet jointly with the global context, $h_i = \phi(z_i, g_t)$; symmetric pooling gives $\bar{h} = \rho(\{h_j\}_{j=1}^k)$ (mean pooling). Per-wallet settle and flush logits are then

$$\ell_i^s = \psi_s(h_i, \bar{h}, g_t), \quad \ell_i^f = \psi_f(h_i, \bar{h}, g_t, e(a_s)),$$

while the no-op logits are produced from global context ($\bar{h}, g_t$) alone (for SC-FAC, $e(a_s)$ is the gathered feature h_{a_s} , zero for $a_s = \perp$). Because $\phi, \rho, \psi_s, \psi_f$ are shared across wallets and no tensor is sized by k , the map is permutation-equivariant and its weights can be loaded and executed at any wallet count without retraining—the property a flat MLP lacks (Fig. 2).

4. EXPERIMENTS AND RESULTS

All policies are evaluated deterministically on fixed, shared test pools; model and training choices use validation only. Inference uses paired t tests on matched seeds, and we additionally report Holm step-down adjusted p -values within pre-specified contrast families; these corrections are exploratory, as the study was not powered for them. Rule baselines are the native one-flush references FA and FWF (reconstructed, since the reference design allowed multiple flushes per step), a rotation rule (ROT), and the balance-feedback policy **BFP**: best-fit settlement that proactively flushes the fullest usable wallet when its remaining balance falls below a fraction of wallet capacity. Its single threshold (0.5) was chosen once on the 300-episode validation pools by mean Money over the implemented candidate thresholds $\{0.3, 0.5, 0.8\}$, then frozen and evaluated once on the test pools; it was not re-tuned per capacity. We also report an oracle feasible value and the drop decomposition.

Factorization and the strong baseline. Table 1 reports Money per capacity and Table 2 the seed-level paired contrasts at $C=1200$. Every method incurs the same structural oversize floor of about 400 drops per episode; the policy differences lie in the avoidable fraction, which at $C=1200$ falls from 77.0 drops/episode for JA-PPO to 49.5 (IFAC) and 40.4 (SC-FAC). *Factorization is the reliable learned effect.* SC-FAC outperforms JA-PPO at all four capacities (+677, +999, +1327, +1285 Money; $p = 0.012, 0.0067, < 0.001, 0.004$; 5/5 seeds), and all four contrasts survive Holm correction (adjusted $p = 0.036, 0.027, 0.0003, 0.022$). IFAC outperforms JA-PPO at $C \in \{800, 900, 1200\}$ in raw tests ($p < 0.001, 0.032, < 0.001$) but not at $C=1000$ ($p = 0.57$); after Holm correction only $C \in \{800, 1200\}$ remain significant ($C=900$ adjusted $p = 0.064$). The added settle-conditioning path does not account for this gain: the SC-FAC–IFAC contrast includes zero at $C \in \{900, 1000, 1200\}$, and at $C=800$ SC-FAC is lower ($-419, p = 0.009$). The parameterization itself is smaller as well as stronger: outputs drop from 625 to 50 logits and the actor from 245,874 to 98,099 parameters (IFAC); the set variants use about 135k parameters with no k -dependent tensor and sub-millisecond per-step CPU latency. *The feedback rule sets the level to beat.* BFP attains the highest Money at every capacity with zero avoidable drops, and its advantage is significant in 11 of 12 method-capacity paired contrasts after Holm correction; the sole exception is versus IFAC at $C=1000$ (+2545, $p = 0.057$, 95% CI including zero), which BFP nevertheless wins on all five seeds.

Settle-conditioning mechanism. Table 3 compares SC-FAC with the no-conditioning and shuffled-conditioning variants under matched parameters, seeds, and budget at $C=1200$; the corresponding paired intervals all include zero (-284 with CI $[-1678, 1110]$, $p = 0.47$; -301 with CI $[-1212, 611]$, $p = 0.29$), as do the matched contrasts at $C=800$ (raw $p = 0.27$ and 0.65); all Holm-adjusted values are 1.0. The fully conditioned variant also spends

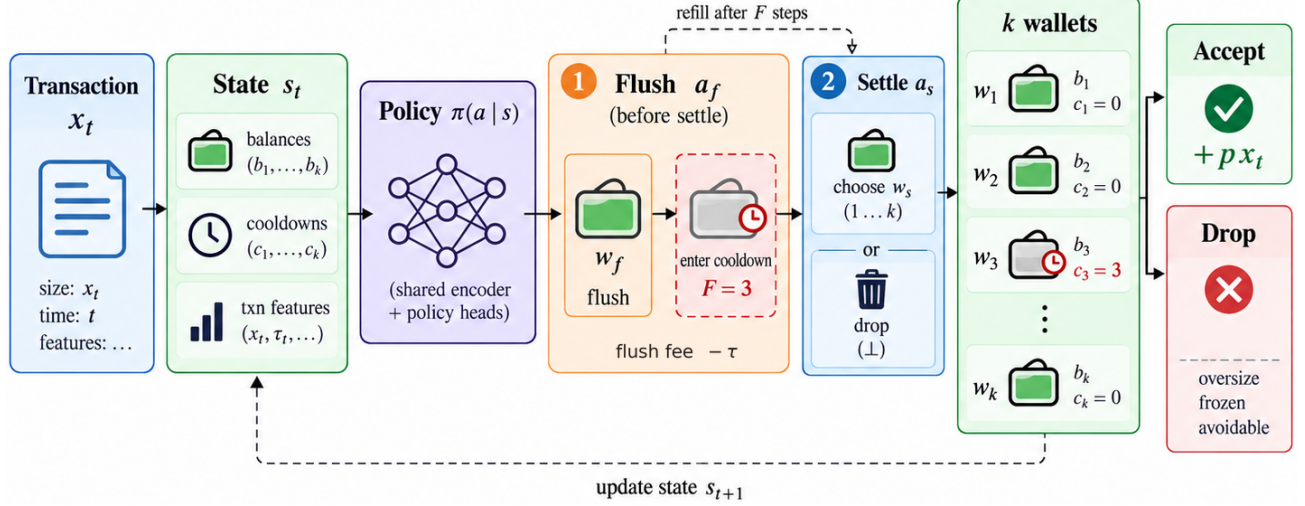


Fig. 1. Streaming collateral control. At each arrival the controller settles x_t into one of k capacity-partitioned wallets or drops it, and may flush one wallet; the flush executes before settlement, incurs fee τ , and starts a cooldown of F steps. A drop carries no additional penalty; drops are oversize, frozen, or avoidable. The binary availability indicator in the $3k+2$ observation is omitted from the state panel for compactness.

method	$C = 800$	$C = 900$	$C = 1000$	$C = 1200$
FA	2907	4172	5533	8826
FWF	2312	3482	4713	7640
BFP0.5 (rule)	4682	7027	9497	15327
JA-PPO	3080±144	4405±185	6352±36	11893±149
IFAC	4176±69	5465±188	6952±960	13514±96
SC-FAC	3757±43	5404±104	7680±45	13179±259

Table 1. Money per episode (mean±SE over $n=5$ training seeds) versus collateral C . Rules need no seeds.

contrast (a-b)	Δ Money	95% CI	p	wins/ n
SC-FAC-IFAC	-336	[-1231,560]	0.357	1/5
IFAC-JA-PPO	1621	[1108,2134]	< 0.001	5/5
SC-FAC-JA-PPO	1285	[671,1900]	0.004	5/5
BFP0.5-SC-FAC	2148	[1428,2868]	0.001	5/5
BFP0.5-IFAC	1813	[1546,2079]	< 0.001	5/5
BFP0.5-JA-PPO	3433	[3020,3846]	< 0.001	5/5
SC-FAC-FA	4353	[3633,5072]	< 0.001	5/5
IFAC-FA	4688	[4421,4955]	< 0.001	5/5

Table 2. Seed-level paired Money differences at $C=1200$ (mean $a - b$, paired- t 95% CI, p -value, wins over seeds; statistical unit is the matched training seed, $n=5$). Positive values favor a .

marginally more on flushes without higher accepted value. At the tested flush price and observation model, the identity of the planned settle wallet carries no detectable additional decision-relevant signal once wallet state is observed.

variant	Money ($C=1200$, same 3 seeds)	flush cost
SC-FAC	12951±399	571
SC-FAC (no cond.)	13235±100	545
SC-FAC (shuf. cond.)	13252±217	579

Table 3. Settle-conditioning mechanism ablations ($C=1200$; same $n=3$ seeds per variant).

Cross- k transfer and regime shifts. We train at $k \in \{6, 12, 24\}$ and deploy, with no retraining, at every k . Fig. 3 shows that the set encoder loads and runs at every deployment k , including off-diagonal cells where a flat policy has no entry because its weight shapes change with k . The benefit is operational deployability rather

than uniformly preserved performance: zero-shot retention is asymmetric, strongest toward larger-wallet operating points and weakest when a model trained with large wallets meets the small wallets at $k=24$ (e.g. $k=6 \rightarrow 24$ yields 4433). At matched k the set encoder is slightly above the flat MLP at $k=6$ (46742 vs. 44820) and $k=12$ (36391 vs. 34193); at $k=24$ the all-seed aggregate is 9190 vs. 13610, and it includes one failed training seed (seed 532, Money 0, retained in the primary aggregate). Excluding that single failure gives 13785 vs. 13610 ($n=3$); the all-seed result is therefore sensitive to one training failure, and an exclusion sensitivity analysis is reported separately rather than used as the headline.

For regime shifts, every policy is trained on stationary pools only and tested on identical deterministic episodes spliced across regimes at an abrupt change point (six scenarios [16]: calm $\leftrightarrow$ burst, early and late). Table 4 reports post-switch avoidable drops and Money. BFP incurs zero avoidable drops after every switch and the highest Money, retaining its behavior without retraining. The learned policies transfer substantially better than the naive rule references (28–56 versus 144–148 post-switch drops). Against BFP, JA-PPO and IFAC remain worse on both metrics over the six paired scenarios (one-sample paired tests, $n=3$; Holm-adjusted $p \leq 0.026$); for SC-FAC the drops contrast is not significant ($p = 0.088$), and its raw Money contrast ($p = 0.035$) does not survive Holm correction ($p = 0.069$). We therefore observe robustness of the learned policies to the stream change, but no switch-adaptation advantage over the tuned feedback rule.

policy	Money	post Money	post rec. drops
FA	8585	4232	143.8
FWF	7593	4075	148.4
BFP0.5 (rule)	15319	8206	0.0
JA-PPO	11694	6114	56.0
IFAC	13681	7220	28.3
SC-FAC	13108	7006	28.3

Table 4. Switching streams (mean over six scenarios; learned policies over three seeds): total Money, post-switch Money, and post-switch recoverable drops.

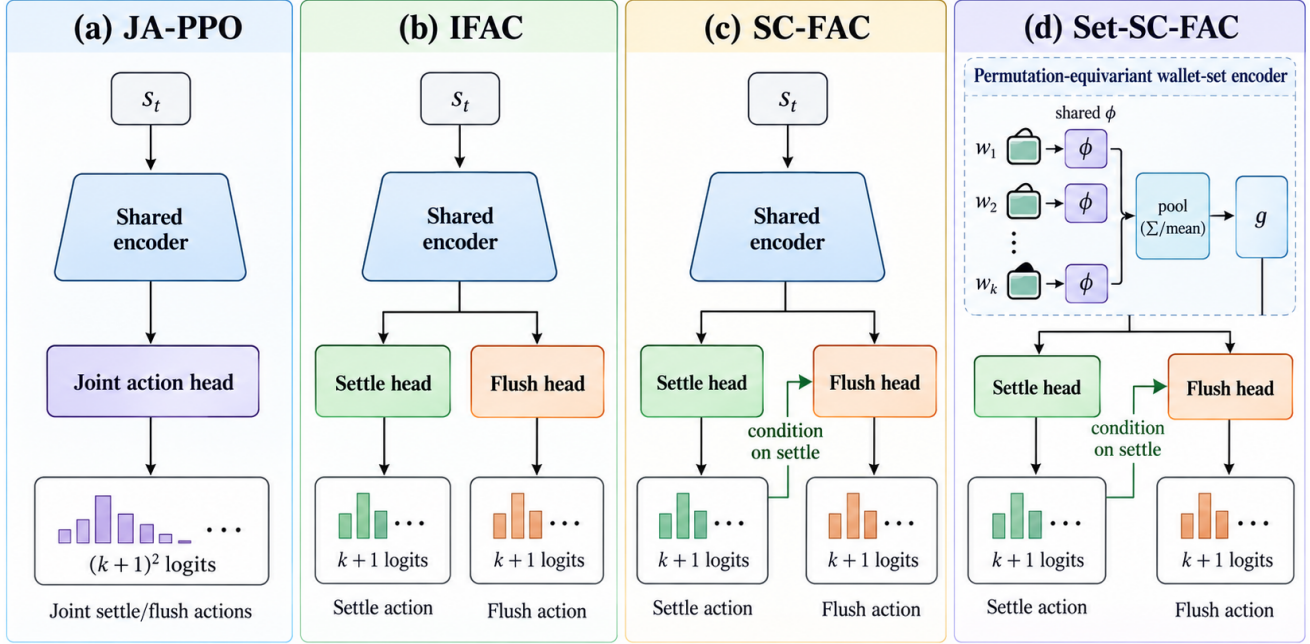


Fig. 2. Action parameterizations: (a) JA-PPO, one joint head over $(k+1)^2$ outcomes; (b) IFAC, independent settle and flush heads of $k+1$ logits each; (c) SC-FAC, the flush distribution is conditioned on the sampled settle action a_s ; (d) Set-SC-FAC, the k -shaped trunk is replaced by a shared per-wallet encoder with symmetric pooling. Conditioning denotes a policy-level dependency; environment execution remains flush-before-settle.

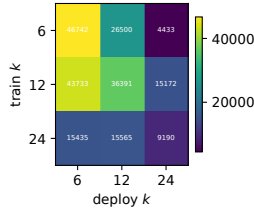


Fig. 3. Cross- k transfer matrix (Set-SC-FAC): rows train k , columns deploy k . Off-diagonal cells are zero-shot deployments; a flat MLP has no off-diagonal entries.

5. DISCUSSION AND CONCLUSION

Factoring the joint action into separate settle and flush heads is the reproducible structural improvement in this study: gains over the quadratic joint head survive multi-seed paired tests at every capacity for SC-FAC and at two of four capacities for IFAC after Holm correction, coincide with a direct reduction in avoidable drops, and reduce outputs from $(k+1)^2$ to $2(k+1)$ and the actor to roughly 40% of its parameters. In contrast, the settle→flush conditioning path shows no detectable Money benefit under zero, shuffled, and matched-seed comparisons at the tested flush price.

The permutation-equivariant encoder provides a different property: operational cross- k deployability, allowing one trained policy to be loaded at wallet counts the training never saw. Zero-shot retention is asymmetric, and the all-seed matched- k comparison at $k=24$ is dominated by one failed training run rather than a stable performance deficit. A practitioner fixed at one wallet count gains

little from the set parameterization; one who must redeploy across changing counts gains portability that a fixed-shape network cannot provide.

The validation-selected feedback rule is the strongest controller tested, on stationary streams and after abrupt switches. On this homogeneous, full-observation, i.i.d.-to-bursty workload, observable wallet fill therefore captures much of the decision-relevant information. This bounds the setting in which the learned policies have demonstrated an advantage; settings in which richer temporal prediction is genuinely necessary—predictable non-i.i.d. burst structure, heterogeneous wallet sizes or fees, multi-step lookahead under expensive flushes—are the natural direction for future work. Limitations are one flush per step, a finite horizon, one fixed fee, no lookahead, homogeneous wallets, single-process PPO without vectorized rollouts, and training instability at some k (the retained failed seed). The environment is a faithful port of the prior one, and policies together with unspecified hyperparameters are documented re-implementations selected on validation; the prior manuscript’s learned numbers are treated as reported rather than regenerated.

Acknowledgment

Generative AI assistance (OpenAI ChatGPT) was used to support manuscript drafting and editing and to produce the two conceptual illustrations, and an AI coding assistant was used during code development and visualization. All experimental results, statistical analyses, and technical claims were produced by the released code with fixed seeds and evaluation pools, and were reviewed and approved by the authors.

6. REFERENCES

- [1] Joseph Poon and Thaddeus Dryja, “The bitcoin lightning network: Scalable off-chain instant payments,” Tech. Rep., Technical Report, 2016.
- [2] Andrew Miller, Iddo Bentov, Surya Bakshi, Ranjit Kumaresan, and Patrick McCorry, “Sprites and state channels: Payment networks that go faster than lightning,” in *Financial Cryptography and Data Security (FC 2019)*. 2019, vol. 11598 of *Lecture Notes in Computer Science*, pp. 508–526, Springer.
- [3] Kartick Kolachala, Mohammed Ababneh, and Roopa Vishwanathan, “Sok: Payment channel networks,” *arXiv preprint arXiv:2407.20968*, 2024.
- [4] Vibhaalakshmi Sivaraman, Shaileshh Bojja Venkatakrishnan, Kathleen Ruan, Parimarjan Negi, Lei Yang, Radhika Mittal, Giulia Fanti, and Mohammad Alizadeh, “High throughput cryptocurrency routing in payment channel networks,” in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2020)*. 2020, pp. 777–796, USENIX Association.
- [5] Allan Borodin and Ran El-Yaniv, *Online Computation and Competitive Analysis*, Cambridge University Press, 1998.
- [6] Thodoris Lykouris and Sergei Vassilvitskii, “Competitive caching with machine learned advice,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 2018, vol. 80 of *Proceedings of Machine Learning Research*, pp. 3296–3305, PMLR.
- [7] Ghada Almashaqbeh, Sixia Chen, and Alexander Russell, “Competitive policies for online collateral maintenance,” in *6th Conference on Advances in Financial Technologies (AFT 2024)*, 2024, vol. 316 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pp. 26:1–26:16.
- [8] Qi Lin, Hongjian Yi, Jiashu Pang, Minghua Chen, Adam Wierman, Michael Honig, and Yuanzhang Xiao, “Competitive online optimization under inventory constraints,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems (POMACS)*, vol. 3, no. 1, pp. 1–28, 2019.
- [9] Arash Tavakoli, Fabio Pardo, and Petar Kormushev, “Action branching architectures for deep reinforcement learning,” in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. 2018, vol. 32, pp. 4131–4138, AAAI Press.
- [10] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Póczos, Ruslan Salakhutdinov, and Alexander J. Smola, “Deep sets,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [12] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *arXiv preprint arXiv:1506.02438*, 2015.
- [13] Gabriel Dulac-Arnold, Richard Evans, Hado van Hasselt, Peter Sunehag, Timothy Lillicrap, Jonathan Hunt, Timothy Mann, Theophane Weber, Thomas Degris, and Ben Coppin, “Deep reinforcement learning in large discrete action spaces,” *arXiv preprint arXiv:1512.07679*, 2015.
- [14] Matthew Hausknecht and Peter Stone, “Deep reinforcement learning in parameterized action space,” in *International Conference on Learning Representations (ICLR)*, 2016.
- [15] Fabian Akkerman, Julius Luy, Wouter van Heeswijk, and Maximilian Schiffer, “Dynamic neighborhood construction for structured large discrete action spaces,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [16] Joao Gama, Indre Zliobaite, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.